

Cours de Git/Git hub

1. Qu'est-ce que Git ?

Explication :

- **Git est un système de contrôle de version distribué.**
 - Cela signifie qu'il permet de suivre les modifications apportées aux fichiers au fil du temps.
 - Chaque développeur travaille avec une copie complète du projet sur sa machine, ce qui permet de travailler hors ligne et de partager les modifications plus tard.

Pourquoi c'est important :

- Git garde un historique de toutes les modifications apportées aux fichiers.
- Si vous faites une erreur, vous pouvez revenir à une version précédente du projet.
- Il facilite la collaboration entre plusieurs développeurs sur un même projet.

2. Pourquoi utiliser Git ?

Explication :

1. Garder un historique des modifications :

- a. Git enregistre chaque changement apporté aux fichiers, avec un message descriptif (commit).
- b. Vous pouvez voir qui a fait quelle modification et quand.

2. Collaborer efficacement avec d'autres développeurs :

- a. Plusieurs personnes peuvent travailler sur le même projet sans écraser les modifications des autres.
- b. Git permet de fusionner les modifications de manière contrôlée.

3. Revenir à une version précédente en cas de problème :

- a. Si vous introduisez un bug ou si vous voulez annuler des modifications, Git vous permet de revenir à un état précédent du projet.

Exemple concret :

- Imaginez que vous travaillez sur un document texte. Git vous permet de sauvegarder chaque version du document (comme une "photo" du document à un instant donné). Si vous faites une erreur, vous pouvez revenir à une version précédente.

3. Différence entre Git et GitHub

Explication :

1. Git :

- a. C'est un outil **local** que vous installez sur votre ordinateur.
- b. Il gère les versions de votre projet sur votre machine.

2. GitHub :

- a. C'est une plateforme **en ligne** qui héberge des dépôts Git.
- b. Il ajoute des fonctionnalités sociales comme les **pull requests** (demandes de fusion) et les **issues** (suivi des problèmes).
- c. GitHub permet de partager votre projet avec d'autres développeurs et de collaborer à distance.

Analogie :

- Git est comme un outil de bricolage (par exemple, un marteau) que vous utilisez localement.
- GitHub est comme un atelier en ligne où vous pouvez partager vos projets et collaborer avec d'autres bricoleurs.

4. Concepts de base de Git

1. Dépôt (Repository) :

- **Définition :**

- Un dépôt est un projet versionné avec Git.
- Il contient tous les fichiers de votre projet ainsi que l'historique des modifications.

- **Comment ça marche :**

- Vous initialisez un dépôt avec la commande `git init`.

- Cela crée un dossier caché `.git` qui contient toutes les informations sur l'historique du projet.
- **Exemple :**
 - Si vous avez un projet de site web, le dépôt contiendra tous les fichiers HTML, CSS, JavaScript, ainsi que l'historique des modifications.

2. Commit :

- **Définition :**
 - Un commit est un instantané des modifications apportées aux fichiers.
 - Chaque commit a un message descriptif qui explique ce qui a été modifié.
- **Comment ça marche :**
 - Vous ajoutez des fichiers à l'index (staging area) avec `git add`.
 - Vous créez un commit avec `git commit -m "Message descriptif"`.
- **Exemple :**
 - Si vous ajoutez une nouvelle fonctionnalité à votre projet, vous créez un commit avec un message comme "Ajout de la fonctionnalité de connexion utilisateur".

3. Branche :

- **Définition :**
 - Une branche est une ligne de développement indépendante.
 - Elle permet de travailler sur des fonctionnalités ou des corrections sans affecter la branche principale.
- **Comment ça marche :**
 - Vous créez une nouvelle branche avec `git branch nom-de-la-branche`.
 - Vous basculez entre les branches avec `git checkout nom-de-la-branche`.
- **Exemple :**
 - Si vous travaillez sur une nouvelle fonctionnalité, vous créez une branche appelée `feature/nouvelle-fonctionnalité`. Une fois la fonctionnalité terminée, vous fusionnez cette branche avec la branche principale.

4. Fusion (Merge) :

- **Définition :**
 - La fusion permet de combiner les modifications de deux branches.
 - Par exemple, fusionner une branche de fonctionnalité avec la branche principale.
- **Comment ça marche :**
 - Vous fusionnez une branche avec `git merge nom-de-la-branche`.
 - Si Git détecte des conflits (par exemple, deux branches ont modifié la même ligne d'un fichier), il vous demandera de les résoudre manuellement.
- **Exemple :**
 - Si vous avez terminé une fonctionnalité dans une branche `feature/nouvelle-fonctionnalité`, vous fusionnez cette branche avec `main` pour intégrer les modifications.

Résumé des étapes pour utiliser Git :

1. **Initialiser un dépôt :**
 - a. Créez un dossier pour votre projet.
 - b. Initialisez un dépôt Git avec `git init`.
2. **Ajouter des fichiers :**
 - a. Créez ou modifiez des fichiers dans votre projet.
 - b. Ajoutez les fichiers à l'index avec `git add`.
3. **Effectuer un commit :**
 - a. Créez un commit avec un message descriptif avec `git commit -m "Message"`.
4. **Travailler avec des branches :**
 - a. Créez une nouvelle branche avec `git branch`.
 - b. Basculez entre les branches avec `git checkout`.
 - c. Fusionnez les branches avec `git merge`.
5. **Collaborer avec GitHub :**
 - a. Créez un dépôt sur GitHub.
 - b. Liez votre dépôt local au dépôt distant avec `git remote add origin`.
 - c. Poussez vos modifications avec `git push`.

- Installation de Git

Avant d'utiliser Git, il faut l'installer sur votre machine.

- **Windows** : Télécharger et installer [Git pour Windows](#).
- **macOS** : Utiliser Homebrew : `brew install git`.
- **Linux** : Installer via le gestionnaire de paquets, par exemple :
 - Ubuntu/Debian : `sudo apt install git`
 - Fedora : `sudo dnf install git`

Vérifiez l'installation avec :

```
bash
git --version
```

2. Configuration de Git

Une fois installé, configurez votre identité :

```
bash
git config --global user.name "Votre Nom"
git config --global user.email "votre@email.com"
```

Vous pouvez voir votre configuration avec :

```
bash
git config --list
```

3. Initialiser un Dépôt Git

Un dépôt Git est un projet versionné. Pour en créer un :

```
bash
git init
```

Cela crée un dossier caché `.git` contenant l'historique des modifications.

4. Ajouter des fichiers à Git

Pour suivre un fichier avec Git :

```
bash
git add nom_du_fichier
```

Ou pour ajouter tous les fichiers :

```
bash
git add .
```

5. Faire un Commit

Un commit est un instantané des modifications. Après avoir ajouté des fichiers, enregistrez-les avec :

```
bash
git commit -m "Message décrivant les modifications"
```

6. Vérifier l'État du Dépôt

Avant de committer, il est utile de voir l'état du projet :

```
bash
git status
```

7. Consulter l'Histoire des Commits

Pour voir l'historique des modifications :

```
bash
git log
```

Avec une version plus compacte :

```
bash
git log --oneline --graph --decorate
```

8. Créer et Gérer des Branches

Les branches permettent de travailler sur différentes versions du projet.

Créer une nouvelle branche :

```
bash
git branch nom_de_la_branche
```

Passer à une autre branche :

```
bash
git checkout nom_de_la_branche
```

Ou en une seule commande :

```
bash
git checkout -b nouvelle_branche
```

Voir toutes les branches :

```
bash
git branch
```

9. Fusionner des Branches (Merge)

Quand une branche est prête, elle peut être fusionnée dans la branche principale (main ou master) :

```
bash
git checkout main
git merge nom_de_la_branche
```

10. Annuler des Modifications

- **Annuler un git add :**

bash

```
git reset nom_du_fichier
```

- **Annuler le dernier commit (tout en gardant les modifications) :**

bash

```
git reset --soft HEAD~1
```

- **Revenir à un commit précédent :**

bash

```
git checkout ID_du_commit
```

Étape 3 : Créer un premier dépôt

Ce que vous devez dire :

1. Initialiser un dépôt local :

- a. "Nous allons créer un dossier pour notre projet et l'initialiser avec Git."

2. Ajouter des fichiers :

- a. "Nous allons créer un fichier README.md et l'ajouter à l'index (staging area)."

3. Effectuer un commit :

- a. "Un commit est un instantané des modifications. Nous allons créer notre premier commit avec un message descriptif."

4. Voir l'historique des commits :

- a. "Nous pouvons voir l'historique des commits avec la commande git log."

Exercices pratiques Ce que vous devez dire :

- "Maintenant, c'est à vous de jouer ! Créez un dépôt local, ajoutez des fichiers, et effectuez plusieurs commits."

